R11 — A memória que sobrevive

O alvo que volta depois de desligar, e o barramento de dois fios que o leva para fora do chip

Microcontroladores — DENE/UFMT

OBJETIVOS

- Guardar o alvo na memória não volátil interna e reconhecer uma memória que nunca foi gravada.
- Decidir quando gravar, com a conta da vida útil da memória.
- Medir no osciloscópio uma transação I²C: a subida lenta, o reconhecimento e o byte de controle.
- Comparar escrita byte a byte e por página, e memória interna com externa.

NESTE KIT

Chave	Posição
CH2-1 (LCD)	ON
Botões	`INT0` (SW12, RB0) sobe 0,5 °C e `INT1` (SW13, RB1) desce — **só na Parte 1**
Chave da memória I²C	ON **só na Parte 2** — confirme qual é no seu kit
CH5 (relés), SWITCHS e CH3	OFF

ATENÇÃO

RB0 e RB1 têm dois donos (aula 11): na Parte 1 são os botões; na Parte 2 são SDA e SCL do barramento. Não aperte os botões durante a Parte 2.

Este roteiro grava memória de verdade, e memória se gasta. Use o endereço que o professor indicar para o seu grupo, e nunca deixe um programa gravando num laço.

Pontuação. As seis tarefas somam 8,0. As previsões P1 a P5 da aula 11, preenchidas antes da sessão, valem 0,4 cada. A nota de cada tarefa e de cada previsão está na margem. A seção *Se sobrar tempo* não vale nota.

1 Previsões — entregues no início da sessão

P1. Escreva o alvo na memória, desligue a placa, ligue de novo. O que espera ver no display no primeiro segundo? PREVISÃO · **0,4 pt**

P2. No osciloscópio, o tempo de subida de SDA é igual ao de descida? PREVISÃO · **0,4 pt**

P3. Gravando oito bytes um a um e depois os mesmos oito por página, que razão entre os dois tempos você espera? PREVISÃO · **0,4 pt**

P4. Se o programa gravasse o alvo a cada volta do laço, em quanto tempo a posição se esgotaria? PREVISÃO · **0,4 pt**

P5. Uma escrita na memória interna e uma na externa: qual é mais rápida, e por quê? PREVISÃO · **0,4 pt**

2 Parte 1 — a memória interna

O alvo fica em três bytes a partir de `ENDERECO`: um **marcador** (`0xA5`) e os dois bytes do alvo em décimos. Sem o marcador, o programa não teria como distinguir um alvo gravado de uma memória apagada, que lê `0xFF` em toda posição.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>
#include "lcd.h"

#define ENDERECO      0x10u     /* o professor indica um por grupo          */
#define MARCADOR      0xA5u
#define ALVO_PADRAO   400       /* 40,0 graus                                */
#define QUIETO_MS     3000u     /* grava depois de 3 s sem mexer (Tarefa 2) */

/* ---- memoria interna (aula 11) ---- */
static uint8_t eeprom_ler(uint8_t end)
{
    EEADR = end;
    EECON1bits.EEPGD = 0;  EECON1bits.CFGS = 0;
    EECON1bits.RD = 1;
    return EEDATA;
}

static void eeprom_escrever(uint8_t end, uint8_t dado)
{
    EEADR = end;  EEDATA = dado;
    EECON1bits.EEPGD = 0;  EECON1bits.CFGS = 0;
    EECON1bits.WREN = 1;
    uint8_t salvo = INTCONbits.GIE;
    INTCONbits.GIE = 0;                 /* a fechadura nao pode ser interrompida */
    EECON2 = 0x55;
    EECON2 = 0xAA;
    EECON1bits.WR = 1;
    INTCONbits.GIE = salvo;
    while (EECON1bits.WR) { }           /* cerca de 4 ms */
    EECON1bits.WREN = 0;
}

static int16_t alvo_ler(void)
{
    if (eeprom_ler(ENDERECO) != MARCADOR) {
        return ALVO_PADRAO;             /* nunca gravada: 0xFF */
    }
    return (int16_t)(((uint16_t)eeprom_ler(ENDERECO + 1u) << 8)
                     | eeprom_ler(ENDERECO + 2u));
}

static void alvo_gravar(int16_t alvo)
{
    LATEbits.LATE2 = 1;                 /* marca a gravacao no osciloscopio */
    eeprom_escrever(ENDERECO + 1u, (uint8_t)((uint16_t)alvo >> 8));
    eeprom_escrever(ENDERECO + 2u, (uint8_t)alvo);
    eeprom_escrever(ENDERECO, MARCADOR);    /* por ultimo: so vale se o resto foi */
    LATEbits.LATE2 = 0;
}
```

O programa principal reaproveita a base de 5 ms e o filtro de repique do R7, agora para os dois botões, e mostra o alvo e o número de gravações:

```
/* do R7: tratamento com base de 5 ms e filtro de confirmacao para RB0 e RB1,
   que produz os eventos 'subir' e 'descer'; e ms_ler(). */

void main(void)
{
    int16_t  alvo;
    uint16_t gravacoes = 0, ultimo_toque = 0;
    uint8_t  pendente = 0;

    ADCON1 = 0x0F;                      /* RB0/RB1 e RE digitais */
    LATEbits.LATE2 = 0;  TRISEbits.TRISE2 = 0;
    lcd_iniciar();
    base_e_botoes_iniciar();            /* R7 */

    alvo = alvo_ler();                  /* o primeiro segundo da P1 */

    for (;;) {
        if (subir)  { subir = 0;  if (alvo < 500) { alvo += 5; }  pendente = 1;  ultimo_toque =
ms_ler(); }
        if (descer) { descer = 0; if (alvo > 300) { alvo -= 5; }  pendente = 1;  ultimo_toque =
ms_ler(); }

        if (pendente && (uint16_t)(ms_ler() - ultimo_toque) >= QUIETO_MS) {
            alvo_gravar(alvo);          /* so quando o alvo parou de mudar */
            gravacoes++;
            pendente = 0;
        }

        lcd_posicao(0, 0);  lcd_texto("Alvo ");  lcd_decimos(alvo);
        lcd_posicao(1, 0);  lcd_texto("Gravacoes ");  lcd_numero(gravacoes, 4);
    }
}
```

Tarefa 1. Grave o programa e anote o alvo que aparece. Mude o alvo para 42,5 °C pelos botões, espere o contador de gravações subir, desligue a placa e ligue de novo. TAREFA · **1,5 pt**

Alvo na primeira vez que o programa rodou
Alvo depois de desligar e ligar
Previsão (P1)

(a) Por que o primeiro alvo foi o padrão, e não um número qualquer?

(b) Por que o marcador é o **último** byte gravado?

Tarefa 2. Meça RE2 no osciloscópio durante uma gravação. Depois aperte o botão dez vezes seguidas, rápido, e veja quantas gravações o contador registra. TAREFA · **1,5 pt**

Duração de uma gravação do alvo (3 bytes)
Gravações depois de dez toques rápidos

(a) Com 100 000 escritas por posição, quanto tempo duraria a memória se o programa gravasse a cada toque, para um usuário que ajusta o alvo 20 vezes por dia? E a cada volta do laço (P4)?

(b) O que a espera de 3 s sem mexer compra, e o que ela arrisca?

3 Parte 2 — a memória externa, por dois fios

Solte os botões, ligue a chave da memória I²C e grave o programa de teste. Ele repete a cada 2 s o teste escolhido em `TESTE`, com RE2 em nível alto enquanto a operação dura.

```
#define MEM   0xA0u     /* 1010 000 0: familia, chip 000, escrita */
#define TESTE 1         /* 1: um byte; 2: oito um a um; 3: oito por pagina; 4: endereco errado
*/

static void i2c_iniciar(void)
{
    TRISBbits.TRISB0 = 1;  TRISBbits.TRISB1 = 1;    /* o modulo assume SDA e SCL */
    SSPSTAT = 0x80;                                  /* 100 kHz                   */
    SSPADD  = 39;                                    /* 16 MHz / (4 x 100 kHz) - 1 */
    SSPCON1 = 0x28;                                  /* ligado, mestre I2C        */
    SSPCON2 = 0x00;
}
static void i2c_ocioso(void)  { while ((SSPCON2 & 0x1F) || (SSPSTAT & 0x04)) { } }
static void i2c_start(void)   { i2c_ocioso(); SSPCON2bits.SEN  = 1; while (SSPCON2bits.SEN)
{ } }
static void i2c_restart(void) { i2c_ocioso(); SSPCON2bits.RSEN = 1; while (SSPCON2bits.RSEN)
{ } }
static void i2c_stop(void)    { i2c_ocioso(); SSPCON2bits.PEN  = 1; while (SSPCON2bits.PEN)
{ } }

static uint8_t i2c_escrever(uint8_t b)      /* devolve 1 se houve reconhecimento */
{
    i2c_ocioso();  SSPBUF = b;
    while (SSPSTAT & 0x01) { }
    i2c_ocioso();
    return (uint8_t)!SSPCON2bits.ACKSTAT;
}

static uint8_t i2c_ler_ultimo(void)         /* le um byte e responde sem ACK */
{
    i2c_ocioso();  SSPCON2bits.RCEN = 1;
    while (!(SSPSTAT & 0x01)) { }
    uint8_t b = SSPBUF;
    SSPCON2bits.ACKDT = 1;  SSPCON2bits.ACKEN = 1;
    while (SSPCON2bits.ACKEN) { }
    return b;
}

static void mem_esperar(void)               /* pergunta, em vez de esperar 5 ms */
{
    for (;;) {
        i2c_start();
```

```
        uint8_t ack = i2c_escrever(MEM);
        i2c_stop();
        if (ack) { return; }
    }
}

static void mem_escrever(uint8_t end, const uint8_t *d, uint8_t n)  /* n <= 8 */
{
    i2c_start();
    i2c_escrever(MEM);  i2c_escrever(end);
    for (uint8_t i = 0; i < n; i++) { i2c_escrever(d[i]); }
    i2c_stop();
    mem_esperar();
}

static uint8_t mem_ler(uint8_t end)
{
    i2c_start();
    i2c_escrever(MEM);  i2c_escrever(end);
    i2c_restart();                          /* start repetido: uma transacao so */
    i2c_escrever(MEM | 1u);
    uint8_t b = i2c_ler_ultimo();
    i2c_stop();
    return b;
}
```

```
#define END_EXT 0x20u   /* multiplo de 16: a pagina nao da a volta */

void main(void)
{
    const uint8_t d[8] = { 1, 2, 3, 4, 5, 6, 7, 8 };

    ADCON1 = 0x0F;                      /* RB0/RB1 digitais: o PBADEN volta a importar */
    LATEbits.LATE2 = 0;  TRISEbits.TRISE2 = 0;
    lcd_iniciar();
    i2c_iniciar();

    for (;;) {
        uint8_t lido = 0;
        LATEbits.LATE2 = 1;             /* a largura do pulso e o tempo da operacao */
#if TESTE == 1
        mem_escrever(END_EXT, d, 1);
        lido = mem_ler(END_EXT);
#elif TESTE == 2
        for (uint8_t i = 0; i < 8u; i++) { mem_escrever(END_EXT + i, &d[i], 1); }
#elif TESTE == 3
        mem_escrever(END_EXT, d, 8);
#else
        i2c_start();                    /* o erro numero um: 0x50 sem deslocar */
        i2c_escrever(0x50);  i2c_escrever(END_EXT);
        i2c_restart();
        i2c_escrever(0x51);
        lido = i2c_ler_ultimo();
        i2c_stop();
#endif
        LATEbits.LATE2 = 0;
        lcd_posicao(0, 0);  lcd_texto("Lido: ");  lcd_numero(lido, 3);
```

```
        __delay_ms(2000);              /* aqui pode: o programa so repete o teste */
    }
}
```

Tarefa 3. Com `TESTE 1` (escreve um byte e o lê de volta), canal 1 em SCL e canal 2 em SDA, dispare pela descida de SDA com SCL em alto — a condição de início. TAREFA · **1,5 pt**

Primeiro byte depois do início (leia na tela)
Tempo de subida de SDA (10% a 90%)
Tempo de descida de SDA
Previsão (P2)

(a) Onde está o reconhecimento na tela, e quem o produz?

(b) Por que a subida é lenta e a descida rápida?

Tarefa 4. Meça a largura do pulso em RE2 com `TESTE 2` (oito bytes, um por transação) e com `TESTE 3` (os mesmos oito numa página). TAREFA · **1,5 pt**

Oito bytes um a um
Oito bytes numa página
Razão
Previsão (P3)

(a) Por que a razão é perto de 8?

(b) O programa grava a página a partir de um endereço múltiplo de 16. O que aconteceria começando no endereço 14?

Tarefa 5. Compare a gravação de um byte na memória interna (Tarefa 2, dividida por três) com a de um byte na externa (`TESTE 1`). TAREFA · **1,0 pt**

Um byte na interna
Um byte na externa
Previsão (P5)

Por que a externa não é muito mais lenta, apesar do barramento?

Tarefa 6. Grave com `TESTE 4`: o programa envia `0x50` como byte de controle — o endereço de sete bits da memória, sem deslocar. TAREFA · **1,0 pt**

O que a tela mostra no nono pulso de SCL? O que o display mostra como byte lido?

Explique o erro e o valor certo.

4 Armadilhas frequentes

Sintoma	Causa provável
Alvo absurdo na primeira vez	Faltou conferir o marcador: a memória apagada lê `0xFF`
A gravação nunca termina	Interrupção entre `0x55` e `0xAA`: faltou desligar `GIE`
Nenhum reconhecimento no barramento	Chave da memória em OFF, sem resistor de elevação, ou endereço sem deslocar
Programa preso em `mem_esperar`	A memória não responde nunca: mesma causa da linha anterior
Os botões mexem no barramento	RB0/RB1 são SDA/SCL: botões só na Parte 1

5 Entrega

As tabelas das Tarefas 1 a 6, com as respostas escritas nelas, e a captura da Tarefa 3 com o byte `0xA0` identificado bit a bit. TAREFA

6 Se sobrar tempo

E1 — o alvo em dois lugares. Grave o alvo também na memória externa e, na inicialização, compare as duas cópias. Se discordarem, qual você usa? OPCIONAL

E2 — o custo de perguntar. Troque `mem_esperar` por `__delay_ms(5)` e meça de novo a Tarefa 4. Quanto tempo a pergunta economizava? OPCIONAL

No TP, o alvo sobrevive a desligar. No R13, a memória guarda os ganhos e os coeficientes que o grupo vai aprender a partir dos dados do R10. NOTA